

CHA Dynamic Functions and Variation Points Function Guide

User Guide

Copyright

© Ericsson AB 2020, 2025. All rights reserved. No part of this document may be reproduced in any form without the written permission of the copyright owner.

Disclaimer

The contents of this document are subject to revision without notice due to continued progress in methodology, design and manufacturing. Ericsson shall have no liability for any error or damage of any kind resulting from the use of this document.

Trademark List



Contents

1	CHA Dynamic Functions and Variation Points Introduction	1
1.1	Purpose and Scope	1
2	CHA Dynamic Functions Description	2
2.1	Dynamic Functions	2
2.2	Add Dynamic Functions	2
2.3	Dynamic Function Interface	3
2.4	Dynamic Function Rule	3
2.5	Dynamic Function Versioning and Life Cycle Management	7
2.6	Dynamic Function Mapping	9
2.7	Dictionary Parameters	9
2.8	Performance Management	10
3	Enrichment Validation	11
4	Rule Builder Elements	12
4.1	Rule Builder Nodes	12
4.2	Rule Builder Conditions	13
4.3	Rule Builder Modifiers	18
5	Appendix 1	36
5.1	Regular Expressions	36





1 CHA Dynamic Functions and Variation Points Introduction

1.1 Purpose and Scope

The purpose of the CHA Dynamic Functions and Variation Points Function Guide is to provide descriptions and guidelines for how to create and use dynamic functions.



2 CHA Dynamic Functions Description

2.1 Dynamic Functions

Dynamic Functions are common properties of CHA services and are configured in the dynamic functions rule builder.

The list of dynamic functions is found under the **Dynamic Functions** menu option in the **Network Configuration** app.

Use the filter section at the top of the list to narrow down the number of items when looking for a specific dynamic function.

Each dynamic function is shown with its name, state, description, the interface definition and internal parameters. Click on the card in the list to navigate to the page for that dynamic function.

When a dynamic function has been applied the service context(s) where it's used will be listed in the **Used By** list on the **Details** tab.

2.2 Add Dynamic Functions

In this section, new dynamic functions are created using the **Add Dynamic Function** button.

The following table shows an overview of the possible configurations for creating a new dynamic function:

Table 1 Create new Dynamic Functions

Item	Description
Name	The name of the dynamic function. (Mandatory)
Dynamic Function Description	The description of the dynamic function.
Version Description	The version description of the dynamic function. Can be used to describe the difference compared to the previous version.
Proceed to Rule Builder	Saves the dynamic function details and navigates to the rule builder page to configure the rule. For more information on, how to configure the parameters and elements required for enrichment, see <i>Rule Builder</i> .



Click **Save** to save the new dynamic function. The function is added to the dynamic function list view.

2.3 Dynamic Function Interface

A dynamic function requires input data to perform its task. The results of the dynamic function are accessed through the output. The input and the output is the dynamic function *interface*. At least one input and one output must be defined, but there is no upper limit.

Each input and output definition has a name and a data type. The name is shown when the dynamic function is used, as a key to what input is expected and what output is produced. Choosing names with care makes it easier to configure the mapping when the dynamic function is used. The names are also the identifiers within the rule portion of the dynamic function.

To ensure that data is handled correctly, interface parameters must be configured with a data type. Depending on the intended use of the dynamic function, the source data determines the data type selection. The same considerations must be made when configuring the output. It is important to consider any data type restrictions on the intended output destination.

Besides input and output, internal parameters can be configured. Parameters defined as *Internal* do not show up as input or output when the dynamic function is used. Instead they are used inside the rule to hold intermediate results or values converted to another data type.

2.4 Dynamic Function Rule

The logic of the rule is configured in the dynamic function **Rule Builder**. The rule is organized in a tree hierarchy which will be evaluated from top to bottom.

The following types of element are used to construct the rule logic:

- Nodes
 - Creates branches in the rule logic.
- Conditions
 - Compare information to determine the path to take.
- Modifiers
 - Perform actions.

For a complete list of elements, see [Rule Builder Elements](#) on page 12.



A typical dynamic function get some input, evaluate the input values using conditions, and then manipulate the values or produces output using the modifiers.

2.4.1 Rule editor

The rule is created using the editor on the **Rule** page.

Table 2 Rule editor

Area	Description
Drawer	The Library list elements that can be used to build the rule. The elements are grouped into three tabs — Nodes — Conditions — Modifiers Use the filter to search for an element. Click the plus sign to add an element, or drag it onto the canvas. Note: When using the plus sign the element will be placed last in the selected container. Click the Details tab or click an element on the canvas to open the details view for an element.. The Outline tab in the drawer contain a compact representation if the rule structure.
Canvas	Click on an added element on the canvas to edit its configuration in the Details tab of the drawer. The order of nodes and modifiers can be rearranged within their container or moved to another node using drag and drop.

The input data an element acts on is either provided through the interface and internal parameters, or by entering a constant value. Output is always placed in a parameter.



Internal parameters can be used to keep intermediate results inside the rule for use in later conditions or modifiers.

All elements have the following methods of describing what they do.

- Auto-generated description

The auto-generated description is based on the element implementation, for example, `Set Data Modifier`, show this information when configured "Assign 0 to roamingPosition". This semantic representation is available in the Rule Builder GUI as a compact summary of the configuration.

- User-entered description

The user-entered description is intended for a user to enter. Here the intention of the configuration can be described. There are two advantages of entering a brief description. It can be compared with the semantic representation to verify that the intention and the configuration match. It is also useful as documentation for future users in case the rule needs to be updated. For example, the use of zero to represent *not roaming* is not an integrated part of the rule, and not visible, but something the user specifies. By adding this information, the intention of the configuration becomes visible.

The following modifiers are available:

Table 3 Number Normalization Modifiers

Modifier Name	Descriptions
BestMatch modifier	Modifiers to return the best prefix match length between a string and string list.
BitsModifier	Modifier that extracts the bits from a byte location and sets the result value in the data set.
ByteModifier	Modifier that extracts the bits from a byte location and sets the result value in the data set.
ReplaceString modifier	Modifier that replaces string occurrences from the source string, for a regular expression string, and sets the result value in the data set.
SearchAndAssignStringFromMapModifier	Modifier that searches for an entry in the source Map and sets the value fetched from map to Target String in data set.
StringLengthModifier	Modifier to return the length of the given string.

2.4.1.1

Rule Builder Parameters

The following parameter types are available:

- Input Parameters

The following data types are available in the popup window drop-down list:

- String
- IntegerNumber



- LongNumber
- StringList
- IntegerNumberList
- LongNumberList
- Boolean
- Enumerated
- OctetString
- AddressString
- AddressStringList
- Measurement
- **Output Parameters**
 - The following data types are available:
 - String
 - IntegerNumber
 - StringList
 - IntegerNumberList
 - Enumerated
 - DateTime
 - LongNumber
 - LongNumberList
 - AddressStringList
 - AddressString
 - Measurement
 - OctetStringList
 - OctetString
 - Boolean
- **Internal Parameters**



Specifies where the input parameter value must be fetched from. Internal Parameters are temporary within the tree. The following data types are available:

- String
- IntegerNumber
- StringList
- IntegerNumberList
- StringMap
- OctetString
- AddressStringCategory
- LongNumber
- LongNumberList
- Boolean
- AddressString
- OctetStringList
- Measurement

2.5 Dynamic Function Versioning and Life Cycle Management

A new dynamic function is created using the “Add Dynamic Function” option. After creating a new dynamic function specify a name and an optional description. The user can also add the optional version description for the dynamic function, describing the specific aspects of the version of the dynamic function.

Draft Version:

Configure the rule of the dynamic function using the Rule builder and save the rules using the save option. This creates the draft version of the rule. The draft version can be modified, changes are saved in the existing draft version of the Rule.

Published Version:

Once all the changes are completed, publish the rule using the **Publish** button.

**Active Version:**

The published rule version is made available for use in any service context by activating the rule. When used in a service context, the service context will contain a reference to the dynamic function.

Deactivate:

If a dynamic function is not used in any of the service contexts, it is possible to deactivate the dynamic function. (If the dynamic function is no longer needed).

Constraints after the dynamic function is published:

After a dynamic function is published, it is possible to create new versions of the dynamic function to further evolve the logic. However, once published the definition of the input and output parameters cannot be changed. This limitation is placed on the function as it is referred in service context and parameters are already mapped to the current specification of the dynamic function.

Validation on Import:

The validation is performed on import of different versions of a Dynamic function that already exists in the system. See [CHA Import and Export Function Guide](#).

Edit Mode:

If there is a need to add a new parameter on the interface (input or output) the rule builder provides an option to duplicate the dynamic function. This creates a copy in draft state of the existing dynamic function that can be used to add more input and or output parameters to the dynamic function.

Example 1 Modify an existing dynamic function `ProcessLocation`

1. Select the version of `ProcessLocation` to use as baseline.
2. Create the copy using the **Duplicate** option.
3. Modify the copy with new input parameters.
4. Save the dynamic function with a new name.
5. Publish the new dynamic function.
6. Activate the dynamic function to make this variant available for use in service contexts.

Example 2 Correct the logic of the dynamic function `NormalizeRAT`

1. Select the version of `NormalizeRAT` to use as baseline.



2. Create a new version of the dynamic function by selection the **Create New Version** option in the **Actions** menu.

Create a new version of the dynamic function by clicking the **New Version** button.

3. Modify the logic as required.
4. Publish the new dynamic function.
5. Activate the dynamic function to make this available for use in Service Contexts.

Note: After activation the changed logic will take effect in all service contexts where the dynamic function is referred.

2.6 Dynamic Function Mapping

A dynamic function is not evaluated until it has been included in a mapping configuration. There are several places throughout the CHA configuration where a dynamic function can be applied. In most of those places it is mandatory to insert a dynamic function that produces the required value for a function.

The enrichment mapping is where dynamic functions most utilized. It is used to bring information received in a network request or other sources into a Normalized Event Specification (NES) and to populate the response.

Any dynamic function can be selected when enrichment is configured. The choice depends on the wanted result, that is, the value to put in one of the parameters in the selected NES (request) or AVP/SBI parameter (response).

2.7 Dictionary Parameters

The content of the Request Parameter and Response Parameter lists for source or destination selections are based on the protocol specification dictionary connected to the service context.

If the need arises for additional information, a new version of the protocol specification dictionary can be imported and selected instead of the previous one. Once this operation is performed, the new or changed parameters are available for use when mapping input or output.

Note:

In case parameters used in the configuration are missing in the new version of the dictionary, enrichment might produce unexpected results depending on how the logic of the rule is constructed. This is also the case for other created information that is referenced as enrichment input, such as Characteristics, for example after a business configuration rollback.



2.8 Performance Management

Running a dynamic function comes with a small performance cost. To minimize this cost, dynamic function results can be reused by using the **Copy** system function. When a value can be used as is at the destination, this is preferred as it bypasses the dynamic function framework altogether.



3 Enrichment Validation

Enrichment validation is done for NES parameter values of type **Measurement** and **DateTime**. Validation must be performed in run-time, as no values are known at the time of configuration.

For these data types, the following needs to be validated:

Values

RMCA constrains the values of a NES parameter to a list of discrete values or value ranges.

Validation of the value of the NES parameter derived in CHA is done against these discrete values or value ranges. Validation is successful if the parameter value matches any discrete value or falls in any one of these ranges.

Measurement validation is supported for all the NES parameters with the datatype **Measurement**. To validate, the value of the NES parameter with **Measurement** datatype is expected to be **Integer**, **Long**, or **Decimal**.

If a decimal value is produced by an enrichment for a NES parameter with the type **Measurement** of value **Long**, then the fractional part of that value is truncated.

For example, a NES parameter is defined as **Long** in the NES specification, but the CHA enrichment output produces a **Measurement** output value as 15.99 days. The NES output then holds this value as 15 days.

If the value validation fails, the request is rejected with response code, **PARAMETER_FAULT**.

Cardinality

Validation of the cardinality is done only if the value of a NES parameter is of the type List. **Measurement** and **DateTime** cannot hold list values. Hence, there is no possibility for cardinality validation.



4 Rule Builder Elements

A rule is constructed using three types of elements.

- Nodes
- Conditions
- Modifiers

4.1 Rule Builder Nodes

The following node types are available:

- Node

Groups a set of conditions and modifiers.

The Node element does not have any configurable properties.

- IteratorNode

Groups a set of conditions and modifiers and perform the logic configured inside the IteratorNode as many times as the number of items in the selected **Collection**. The IteratorNode will populate the current value into the selected **IterationValue** property and the current index of the iteration into the **IterationIndex** property. These values can be used in conditions and modifiers within the IteratorNode. Once the iteration is done the parameters will retain the last value and are possible to use in elements below the IteratorNode.

Table 4 IteratorNode parameters

Property	Required	Data Type	Constant Allowed	Description
Collection	Mandatory	List of Any	No	The collection to iterate over.
IterationValue	Optional	Any (Must match the data type of the Collection.)	No	Iteration "variable" that points out the current value during the iteration.
IterationIndex	Optional	Integer	No	Iteration "variable"



Property	Required	Data Type	Constant Allowed	Description
				that points out the current index during the iteration. First element will have index 0.

4.2 Rule Builder Conditions

Different types of rule builder conditions are used to configure dynamic functions. For details on a specific rule builder condition, see the subsection where it is described.

4.2.1 BooleanCondition

The `BooleanCondition` element evaluates the combination of the conditions it contain. Conditions are place in a `BooleanCondition` in the same way as in a `Node` element. The result of the `BooleanCondition` logic depend on the operation setting.

Table 5 Operations

Property	Required	Data Type	Constant Allowed	Description
Operation	Mandatory	N/A	N/A	Supported operations — ALL All child conditions must evaluate to True for this condition to evaluate True. — ANY If any child condition



Property	Required	Data Type	Constant Allowed	Description
				evaluates to True, so will this condition. — NOT Inverts the result from the child condition. Note that only one child condition is allowed with this setting.

4.2.2 CompareDataCondition

Compares two compatible data type values with each other using an operator.

When a parameter of the enumerated data type is selected as input parameter it is possible to select an enumerated value from the protocol specification to compare the value with. This is done by selecting **Constant** as **Parameter**, this will open an option to select one of the enumerated protocol parameters. For each parameter its enumeration values will be available under **Enumeration Value**.

Table 6 ComparDataCondition Parameters

Property	Required	Data Type	Constant Allowed	Description
Input parameter name	Mandatory	Any	No	The comparable data value (left value).
Operator	Mandatory	String	No	The operator. See the table below for valid operations.
Parameter	Mandatory	Any	Yes	The data value which the comparable



Property	Required	Data Type	Constant Allowed	Description
				data value should be compared to (right value).

Table 7 CompareDataCondition supported operators

Operator	Valid for the property data type	Description
EQ	Boolean, Object, DateTime, DecimalNumber, IntegerNumber, BigIntegerNumber, LongNumber, DateTimeInterval, TimeInterval, Time, Date	equals
NEQ	Boolean, Object, DateTime, DecimalNumber, IntegerNumber, BigIntegerNumber, LongNumber, DateTimeInterval, TimeInterval, Time, Date	not equal to
Begins_with	StringDataType	begins with
Ends_with	StringDataType	ends with
Contains	StringDataType	contains
Exact_match	StringDataType	exact match with
Begins_with_ignore_case	StringDataType	begins with ignoring case
Ends_with_ignore_case	StringDataType	ends with ignoring case
Contains_ignore_case	StringDataType	contains ignoring case
Exact_match_ignore_case	StringDataType	exact match with ignoring case
LT	DateTime, DecimalNumber, IntegerNumber, BigIntegerNumber, LongNumber, Time, Date	is less than
GT	DateTime, DecimalNumber,	is greater than



Operator	Valid for the property data type	Description
	IntegerNumber, BigIntegerNumber, LongNumber, Time, Date	
LTE	DateTime, DecimalNumber, IntegerNumber, BigIntegerNumber, LongNumber, Time, Date	is less or equal than
GTE	DateTime, DecimalNumber, IntegerNumber, BigIntegerNumber, LongNumber, Time, Date	is greater or equal than

4.2.3 DoesDataExistCondition

DoesDataExistCondition validates if a parameter exists.

Rule Builder elements deal with missing information by default, this condition can be used when the logic should depend on the existences of a value.

Table 8 DoesDataExistCondition properties

Property	Required	Data Type	Constant Allowed	Description
Data	Mandatory	Any	No	A data value referenced by a data definition.

4.2.4 GlobalTableQueryCondition

GlobalTableQueryCondition performs a COBA table look up and returns true if there are any matches found.

To consider while configuring the condition:

- Only strings and integers are supported as keys and as column data types.
- If the **From parameter** check box is not selected, it is possible to select a COBA instance from available instances fetched from COBA. However, if the **From parameter** check box is selected, the RMCA instance value is fetched from a parameter instead. If the selected parameter at runtime does not contain a valid COBA instance value, the default value is used.



- The **Search typeBest match** is not available if the format of the search column is integer, long, IPv4, or IPv6.

Multiple keys (**Column to search**, **Search type** and **Source**) are supported in Global Table Query Condition. Multiple Keys can be added using the **Additional column to search** options available in Global Table Query Condition.

The delete option can be used to remove the already configured key.

Enhancement like multi column search is supported based on activating the internal Feature Control `FeatureMultipleColumnLookupAndOutputInGlobalList`.

The following table shows `GlobalTableQueryCondition` parameters:

Table 9 GlobalTableQuery Parameters

Property	Required	Data Type	Constant Allowed	Description
COBA specification	Mandatory	String	No	Selects which COBA specification to use.
COBA instance	Mandatory	String	No	Selects which instance of selected COBA specification to use.
Wildcard string for search matching any value	Optional	String	No	Value to use as Lookup wildcard.
Column to search	Mandatory	String	No	Displays search column from selected COBA specification. Read-only.
Search type	Mandatory	String	No	Selects type of search.
Source	Mandatory	String, IntegerNumber Fetch from list if index is given.	No	Value to use as search key.

4.2.5

LengthCondition

`LengthCondition` compares the length of String or length of OctetString or size of the List with the configured value.

Table 10 LengthCondition parameters

Properties	Required	Data Type	Constant Allowed	Description
Input Parameter Name	Mandatory	String, OctetString, List	No	The source data



Properties	Required	Data Type	Constant Allowed	Description
Parameter	Mandatory	Integer	Yes	The length to compare against.
Operator	Mandatory	String	No	The operator to use for the comparison.

4.3 Rule Builder Modifiers

Different types of rule builder modifiers are used to configure dynamic functions. For details on a specific rule builder modifier, see the subsection where it is described.

4.3.1 AddStringModifier

`AddStringModifier` adds a string to the source string at the indicated offset and sets the result value in the data set.

Table 11 AddStringModifier parameters

Properties	Required	Data Type	Constant Allowed	Description
Input Parameter Name	Mandatory	String	No	The source string.
String parameter to add	Mandatory	String	Yes	The string to add
Use Index	Optional	Integer	Yes	The offset into the source string where to insert the add string. Must be in the range: $0 \leq \text{index} \leq \text{SourceString length}$. If omitted, the add string will be appended to the source string



Properties	Required	Data Type	Constant Allowed	Description
Output parameter name	Mandatory	String	No	The result string to be set, referenced by a data definition.

4.3.2 AppendToListModifier

`AppendToListModifier` append the value of an input parameter, or a constant, to a list parameter.

Table 12 `AppendToListModifier` Parameters

Property	Required	Data Type	Constant Allowed	Description
Parameter	Mandatory	Any	Yes	Source input parameter or constant
Output Parameter Name	Mandatory	Any	No	The target collection, referenced by a data definition.

4.3.3 BasicMathModifier

`BasicMathModifier` calculates a basic math operation and sets the result value in the data set. The left and right data value comes from either a constant value or a value in the data set. The left, right, and result data types must be compatible with each other.

Note: `DecimalNumber` must conform to the IEEE 754R `Decimal128` format, 34 digits, and a rounding mode of `HALF_EVEN`.

The following tables show the conversion:

Table 13 `BasicMath` Parameters

Property	Required	Data Type	Constant Allowed	Description
<code>LeftValueSource</code>	Mandatory	Numeric	Yes	The left value
<code>RightValueSource</code>	Mandatory	Numeric	Yes	The right value
Operator	Mandatory	N/A	N/A	The operator. See table below for valid operations.
<code>ResultValueTarget</code>	Mandatory	Numeric	No	The result value of the numeric operation, referenced by a data definition.



Table 14 BasicMath Supported Operations

Operation	Description
ADD	Addition
SUB	Subtraction
DIV	Division
MUL	Multiplication
MOD	Modulo

4.3.4 BitMaskModifier

BitMaskModifier applies a `octetString` to the source, mask, and target, and sets the result value in the data set.

The following table shows the conversion:

Table 15 BitMask Parameters

Property	Description	Data Type	Accepted Values	Cardinality
Source	Source	OctetString	Any	Mandatory
Mask	Mask	OctetString	Any	Mandatory
Target	Result	OctetString	Any	Mandatory

Example 3 Input, Mask, Output

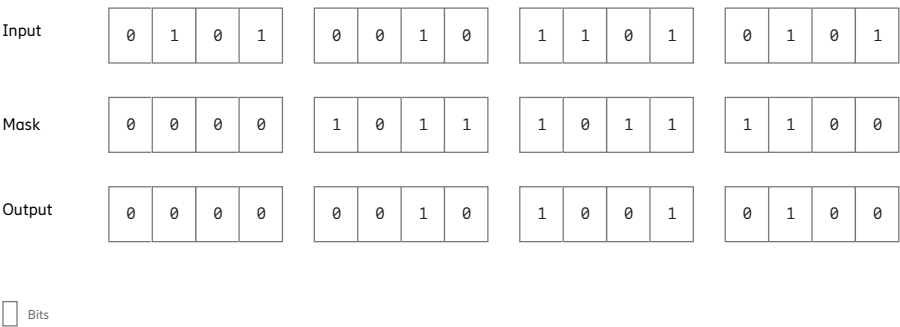


Figure 1 Example Input, Mask, Output

4.3.5 BreakIterationModifier

BreakIterationModifier causes the `IteratorNode` to break the iteration/loop execution. The execution will continue with the next Node.



No properties exist for this modifier.

4.3.6 ConvertDataTypeModifier

`ConvertDataTypeModifier` converts the source to the data type given by the target.

The following data types are supported:

- OctetString to IntegerNumber
- OctetString to String
- IntegerNumber to String
- IntegerNumber to OctetString
- String to IntegerNumber
- String to OctetString

The following table shows the conversion:

Table 16 ConvertDataType Parameters

Property	Description	Data Type	Accepted Values	Cardinality
Source	Source	Any Data Type	Any	Mandatory
Target	Result	Any Data Type	Any	Mandatory

4.3.7 ConvertToMeasurementModifier

`ConvertToMeasurementModifier` converts a value and a unit of measurement (uom) into one single object of type measurement. The modifier creates a measurement object, which contains the value and uom.

Action for converting to measurement datatype. The action consumes a value (as string) and a unit of measurement (uom) (as string) and maps them to a single object of type measurement. For example: consume 1 (value) and usd (uom) are mapped to 1usd of type measurement.

The following table shows the conversion:

Table 17 ConvertToMeasurement Parameters

Property	Description	Data Type	Accepted Values	Cardinality	Support Constant
Value	Value	String	Any	Mandatory	No
UnitOfMeasurement	The unit of Measurement	String	Any	Mandatory	No



Property	Description	Data Type	Accepted Values	Cardinality	Support Constant
Target	Value of the modified value	Measurement	Any	Mandatory	No

4.3.8 CreateAddressStringModifier

CreateAddressStringModifier takes two inputs and combines them to create an addressString.

The following table describes the inputs:

Input Name	Example
-Address Type	URL
-Address Value	http://example.com

The following table shows the creation:

Table 18 CreateAddressString Parameters

Property	Description	Data Type	Accepted Values	Cardinality	Support Constant
SourceCategory	The address type	AddressStringCategory	Any	Mandatory	No
SourceValue	The address value	String	Any	Mandatory	Yes
Target	The AddressString returned	AddressString	Any	Mandatory	No

4.3.9 EnumerationModifier

EnumerationModifier takes a string of key-value pairs separated by two delimiters. One for separating the key-value pairs and the other to separate the key from the value (ValueDelimiter). If no delimiter values are given the default values are used, semicolon (;) for key-value pair separation and colon (:) for separating key and value. The key part of the source string must be an integer since it is matched against the key property in the rule. If the key matches a key from the source, the value part from the source is returned.

Example 4

Source: 1:UTRAN;2:GERAN;3:GAN;4:HSPA Evolution;5:EUTRAN;

FieldDelimiter: ;

ValueDelimiter: :

Key: 3



Result: GAN

If no value is found, the default value is returned. If no default value is given in the `DefaultValue` property, an empty string is returned.

The following table shows the conversion:

Table 19 Enumeration Parameters

Property	Description	Data Type	Accepted Values	Cardinality
Key	Key of the pair key-value	Integer	Any	Mandatory
Source	Source field	String	Any	Mandatory
FieldDelimiter	Field Delimiter	String	Any	Optional
ValueDelimiter	Value Delimiter	String	Any	Optional
DefaultValue	Default value if value does not exist.	String	Any	Optional
Target	Result	String	Any	Mandatory

4.3.10

ExitModifier

`ExitModifier` stops all actions. It halts all further iteration at that point in the decision tree.

4.3.11

ExtractBitsModifier

`ExtractBitsModifier` extracts a set of bits from a large set of octets, by specifying the desired octet and which bits to extract. By default, octets are stored in Big Endian and bits are stored in little Endian. However these are configurable and the conversion is done by `OctetStringEndiannessModifier`.

The following table shows the conversion:

Table 20 ExtractBits Parameters

Property	Description	Data Type	Accepted Values	Cardinality
Source	Source field to extract data.	OctetString	Any	Mandatory
OctetIndex	Octet desired	Integer	Any	Mandatory
BitsIndex	Bits to extract from the Octet.	List<Integer>	Any	Mandatory
Target	Result	OctetString	Any	Mandatory

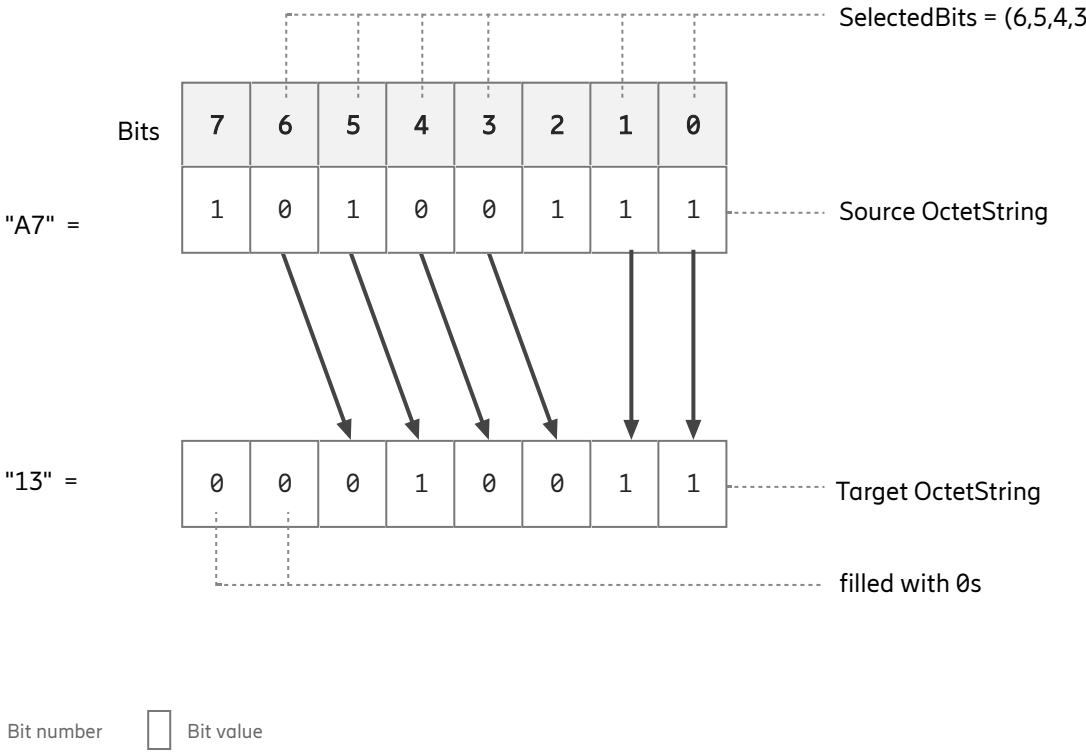


Figure 2 Example of Octet Storing - 1

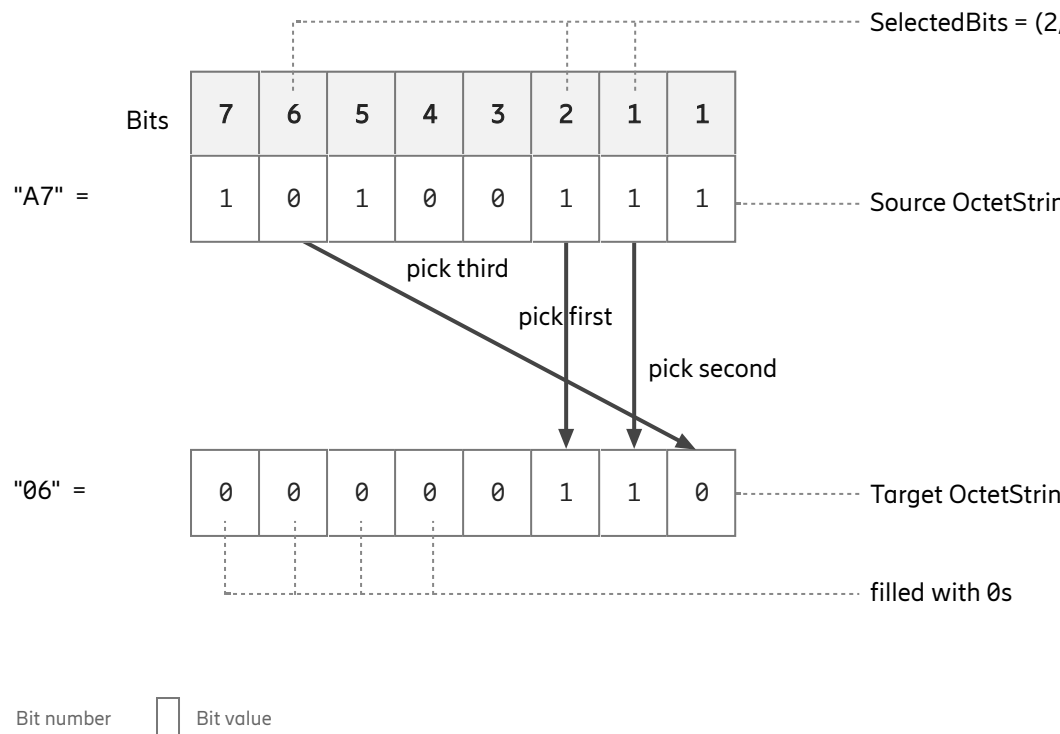


Figure 3 Example of Octet Storing - 2

Note: Extract bit 7,6 and 5 from octet number 10xA15900504707 => 1010
0001 0101 1001 0000 0000 0101 0000 0100 0111 0000 0111

Since Octets are stored in big endian form, the octet with index 1 is the second one from the left. However bits are stored in little endian form, so bits 7, 6, and 5 are: 0101 1001. The Result is: 010 => 2

4.3.12

ExtractNTetModifier

ExtractNTetModifier extracts the N bit from the given octet. A series of 8-bit bytes can be converted to a series of 7-bit septets or 4-bit nibbles. Since 7 bits or 4 bits cannot be represented and there are no 7-bit or 4-bit primitives, they are represented as 8-bit bytes. In these 8-bit bytes, the highest value bits, depending on the size of the type, are always zero.

Example 5

7-bit septet - 0xxx xxxx

4-bit nibble - 0000 xxxx



The following table shows the conversion:

Table 21 ExtractNTet Parameters

Property	Description	Data Type	Accepted Values	Cardinality
Source	Source field to extract data.	OctetString	Any	Mandatory
Nbit	Bit to extract from the Octet.	Integer	Any	Mandatory
Target	Result	OctetString	Any	Mandatory

4.3.13 ExtractOctetModifiers

`ExtractOctetModifiers` extracts a subOctet from the source octet at the start index, and sets the result value on the target property.

The following table shows the conversion:

Table 22 ExtractOctet Parameters

Property	Description	Data Type	Accepted Values	Cardinality
Source	Source field to extract data.	OctetString	Any	Mandatory
StartIndex	Index to begin the extraction.	Integer	Any	Mandatory
LengthToRead	Length of the extraction	Integer	Any	Mandatory
Target	Result	OctetString	Any	Mandatory

4.3.14 GlobalTableQueryModifier

`GlobalTableQueryModifier` performs a COBA table translation. It supports multiple column search based on one or more column values as lookup keys. It also provides wildcard based column value filtering.

To consider when configuring the modifier:

- Only strings and integers are supported as key and as column data types.
- The default value is of datatype *<String>*.
- If the **From parameter** check box is not selected, it is possible to select a COBA instance from available instances fetched from RMCA. However, if the **From parameter** check box is selected, the COBA instance value is fetched from a parameter instead. If the selected parameter at runtime does not contain a valid COBA instance value, the default value is used.
- The **Search typeBest match** is not available if the format of the search column is integer, long, IPv4, or IPv6.



If the number of *Columns to Search* and the *Columns to Return* is 1, the following is applicable:

- If the returned cell is within range, the lower bound value is returned in **Target**. For example, if the cell value includes the range 1–1000, the value 1 is returned. This is only applicable if *FeatureRangeBasedLookupSupportForConfiguredGlobalList* is active, which is connected to the support for evaluation of ranges defined for all Global List by function control c0000046. This is also applicable only if number of *Column To Search* configured is 1.
- The following figure shows the sorting for overlapping ranges:

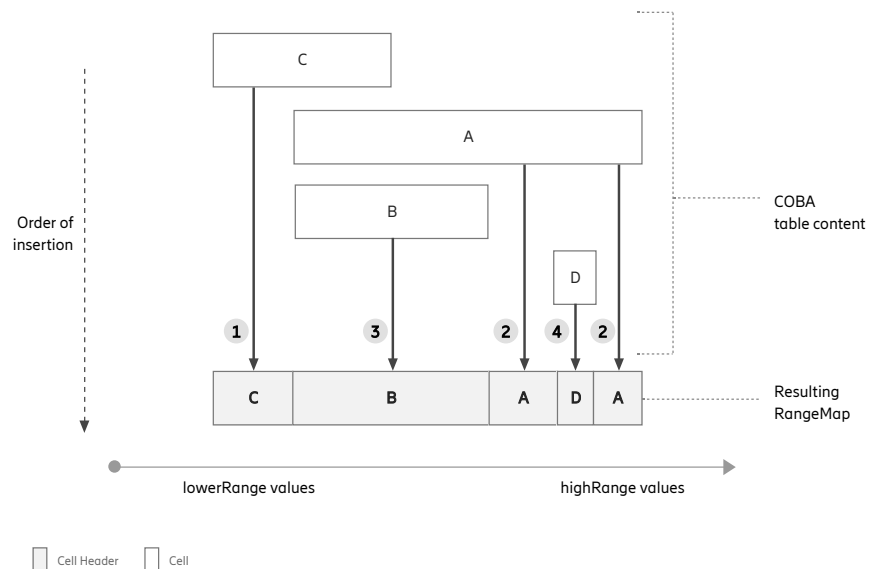


Figure 4 Sorting for Overlapping Ranges

Lowest lower bound ranges are inserted first. However, if lower bound ranges are the same, the highest upper bound range is inserted first.

Ranges added at a later stage overwrite previous underlying ranges.

For multi-column search and return, the following is applicable:

- The *Wildcard string for search matching any value* parameter can be configured so ensure that the row having the configured wildcard value or a particular *columnType* is not filtered out.
- The *Wildcard string for search matching any value* parameter is applicable for columns with *String data Type* and *Search Type* BEST_MATCH only.
- The *Wildcard string for return values to ignore* parameter can be configured such that if the value is present selected by the search filter, it is Ignored and not set to the *Target* property.



Multiple keys (**Column to search**, **Search type** and **Source**) are supported in Global Table Query Modifier. Multiple Keys can be added using the **Additional column to search** options available in Global Table Query Modifier.

Multiple results (**Column to return**, **Target** and **Default value**) are supported in Global Table Query Modifier. Multiple results can be added using the options available in Global Table Query Modifier.

The delete option can be used to remove the already configured key.

Enhancement like multi-column *search and return* with wildcard string is supported based on activating the internal Feature Control `FeatureMultipleColumnLookupAndOutputInGlobalList`.

The following table shows `GlobalTableQueryModifier` properties:

Table 23 GlobalTableQuery Parameters

Property	Description	Data Type	Accepted Values	Cardinality
COBA specification	Selects which COBA specification to use.	String	Values fetched from COBA and presented in drop-down.	Mandatory
COBA instance	Selects which instance of selected COBA specification to use.	String	Values either selected from COBA or from parameter. Values selected from drop-down.	Mandatory
Wildcard string for search matching any value	Value to use as Lookup wildcard.	String	Any	Optional
Wildcard string for return values to ignore	Value to use as Return wildcard.	String	Any	Optional
Column to search	Displays search column from selected COBA specification. Read-only.	String	Not Applicable.	Mandatory
Column to return	Selects which columns value to return.	String	Columns fetched from COBA. Displayed as drop-down.	Mandatory
Search type	Selects type of search.	String	Exact match or best match.	Mandatory
Source	Value to use as search key.	String, IntegerNumber Fetch from list if index is given.	Input, output, or internal parameter matching datatype for column to search.	Mandatory
Target	Parameter to store search result in.	String or IntegerNumber	Output or internal parameter filtered on datatype for column to return.	Mandatory
Default value	Value used when search was not successful.	String	Any.	Optional



4.3.15 IPAddressToStringModifier

`IPAddressToStringModifier` extracts the `OctetString` and a `String` is used as the target, the length of the source then determines what type of address the `OctetString` is converted to.

Example 6

4 octets - IPv4

16 octets - IPv6.

The output is a string.

The following table shows the conversion:

Table 24 `IPAddressToString` Parameters

Property	Description	Data Type	Accepted Values	Cardinality
Source	Source to Convert	OctetString	Any	Mandatory
Target	Result	String	Any	Mandatory

4.3.16 LengthModifier

`LengthModifier` takes a `String`, `OctetString`, or any type that has `DataDefinition` with `CollectionType#LIST`. It returns the length of the provided type according to: `StringDataType` = the number of characters `OctetStringDataType` = the number of octets (bytes) `CollectionType#LIST` = the number of objects in the list

The following table shows the conversion:

Table 25 `Length` Parameters

Property	Description	Data Type	Accepted Values	Cardinality
Source	Source	String/List<Any>	Any	Mandatory
Target	Result	Integer	Any	Mandatory

4.3.17 OctetHexConvertModifier

`OctetHexConvertModifier` takes two parameters, one `String` containing hex (0–9, A–F) and one `OctetString`. Direction of conversion depends on which type is set to be source or target. If source is set to be `String`, the target must be `OctetString` and conversely.

The following table shows the conversion:



Table 26 OctetHexConvert Parameters

Property	Description	Data Type	Accepted Values	Cardinality
Source	Source to convert	String	Any	Mandatory
Target	Result of conversion	OctetString	Any	Mandatory

4.3.18 OctetStringEndiannessModifier

Endianness is the sequential order that bytes are stored in. Big endian sequential order is stored from the large-end first, while, the little endian sequential order is stored from the small-end first. The OctetString modifier inverses this endianness.

4.3.19 ReplaceStringModifier

`ReplaceStringModifier` is used to replace string occurrences in the source string. The substrings that are replaced by the "Regular Expression Constant" are specified in two ways. Either a user defined regular expression in a constant or alternatively provide a string parameter carrying the regular expression. The string to replace the matching substring should be specified by the "Replace Constant", which be a user defined string or a string parameter. The result of the `ReplaceStringModifier` is stored in the "Output Parameter".

For more information, on valid regular expression strings, refer to [Regular Expressions](#) on page 36.

4.3.20 SetDataModifier

`SetDataModifier` Sets a parameter based on a constant or a set parameter. Can also be used to copy a whole list.

Enumerated Datatype is supported.

Enumerated Datatype supports:

- Protocol defined Enumeration (ex: RoamerInOut)
- System defined Enumeration
 - SessionOperationType

When the following settings have been made:

- An Enumerated value is defined as an input parameter.
- A String parameter is defined as an output parameter.



The resulting String output is the Enumerated name of the corresponding Enumerated parameter, as defined in the enrichment function.

Example 7

```
roamerInOut enum definition:  
IN_BOUND(0)  
OUT_BOUND(1)
```

roamerInOut (Enumerated parameter) is chosen as the enrichment input, which is evaluated in SetDataModifier. The enrichment input value received from the request is 0. Then the resulting String output parameter holds the value *IN_BOUND*.

If the enrichment input value received from the request is -1 (unknown enumerated value), then the resulting String output parameter holds an empty string value "".

4.3.21 SplitAddressStringModifier

SplitAddressStringModifier takes in the new AddressStringDataType and splits it in its component values. One string that contains the address value and one AddressStringCategoryDataType that tells what category the address belongs to.

4.3.22 SplitStringModifier

SplitStringModifier is used to split the string "Input parameter" using the delimiter specified by "Constant to Delimit". The modifier tests for substrings matching the "Constant to Delimit" using Regular expression test. Either a user defined regular expression in a delimiter constant or alternatively provide a string parameter carrying the regular expression. The result of the SplitStringModifier is stored in the "Output parameter name" in String List.

For more information, on valid regular expression strings, refer to [Regular Expressions](#) on page 36.

4.3.23 StringPickerModifier

StringPickerModifier picks characters from the source string, in the order given by the position list. The property SourceString contains the string on which the modifier operates. The position ranges from 0 to the length of the source string - 1. The property TargetString must be a data definition that defines where to put the result.



4.3.24 StringToMapModifier

`StringToMapModifier`

The source string is split by delimiters into key-value pairs.

The following table shows the conversion:

Table 27 StringToMap Parameters

Property	Description	Data Types	Accepted Values	Cardinality
Source	Source to convert	String	Any	Mandatory
FieldDelimiter	Field Delimiter	String	Any	Optional
ValueDelimiter	Value Delimiter	String	Any	Optional
Target	Result of conversion	Map <Key-Value>	Any	Mandatory

4.3.25 SubstringModifier

`SubstringModifier` extracts a substring from the source string and sets the result value in the data set.

4.3.26 TbcdToStringModifier

The following table shows the conversion:

Table 28 TbcdToString Parameters

Property	Description	Data Type	Accepted Values	Cardinality
Source	Source	OctetString	Any	Mandatory
Format	TBCD or BCD	String	Any	Mandatory
NibbleSortEnabled	True if the nibbles are swapped.	Boolean	Any	Mandatory
Target	The result	String	Any	Mandatory

4.3.27 TerminateSessionModifier

`TerminateSessionModifier` terminates the ongoing session. The modifier does not require any input parameters and does not produce any output parameter.

4.3.28 DnsLookupModifier

`DnsLookupModifier` performs DNS lookup for the Source string. The modifier sends Source string, DNSType=TXT and DNSClass=ANY to the chosen DNS service to perform the lookup. If the response is not received in the specified time, the lookup is retried for the number of times that is set in the *Number of retries*



parameter. If there is no response received even after all retries, or error response is received, the `DefaultValue` configured in the modifier is set to *Target*. Else, the received response is set to *Target*.

Note: To perform Number Portability lookup using the `DnsLookupModifier` modifier, the source string must contain the B-Party Number in tel URI format.

Table 29 DnsLookupModifier Parameters

Property	Description	DataType	Accepted Values	Cardinality
Service	The DNS services fetched from Service Registry	String	Non-editable drop-down selection	Mandatory
Timeout	Maximum latency for a lookup request. Default = 20 ms	Integer	Any	Mandatory
Number of retries	The number of times a failed lookup request has to be retried. Default = 0	Integer	Any	Mandatory
Source	The B-Party Number in tel URI format to perform the DNS lookup.	String, StringList	Input or internal parameter to perform DNS lookup.	Mandatory
Target	The parameter to store the DNS response.	String, StringList	Any	Mandatory
Default value	The value to put in Target in case the DNS lookup fails.	String	Any	Mandatory

4.3.29

DateTimeModifier

`DateTimeModifier` converts a `dateTime` string to a `DateTime` object.

Using this modifier, a **String** input is validated against a list of supported Joda `DateTime` formats and converted into a `DateTime` output.

If the input **String** is not of the expected format, `CONFIG_ERROR` response code is used to reject the request.

The following table shows the conversion:

Table 30 DateTimeModifier Parameters

Property	Description	DataType	Accepted Values	Cardinality
SourceString	Source to Convert	String	String value of date time	Mandatory
FormatString	Format of output Date Time object	String	Available format in dropdown of modifier	Mandatory



Property	Description	DataType	Accepted Values	Cardinality
TargetDateTime	Result of Date Time Object	DateTime	DateTime output parameter only	Mandatory

If `SourceString` contains the string `1990-09-04T05:59:00Z` and `FormatString` contains `yyyy-MM-dd'T'HH:mm:ssZ`, then `TargetDateTime` will contain the `DateTime` object with `DateTime` value as `1990-09-04T05:59:00.000Z` and `TimeZone` as `UTC` after executing this modifier.

Note: A modifier using a parameter is protected by the condition `DoesDataExistCondition`. This condition checks if the parameter is present.

4.3.30 MapConverterLookupModifier

The dynamic function modifier which can be used to send **REST MNP ATI** request towards MAP Converter for fetching mobile number portability information as a **REST MNP ATI** response.

The modifier takes non-served party's subscriber number, NAI (Nature of Address Indicator) and applicable MAP ATI network profile as input, performs look up towards the MAP Converter and produces the below output values if present in the received REST response.

— Success

- Status
- Routing Number (RN)
- IMSI

— Failure

- Fault Code

— Connection/Request Timeout

- Default values if configured
 - Status
 - Routing Number
 - IMSI

Table 31 MAP Converter Lookup Parameters

Property	Description	DataType	Accepted Values	Cardinality
Subscriber Number	The non-served party subscriber number derived from the traffic request	String	Input or internal parameter	Mandatory



Property	Description	DataType	Accepted Values	Cardinality
Subscriber Number NAI	The non-served party natureOfAddressIndicator derived from the traffic request	Integer	Input or internal parameter	Mandatory
MAP ATI Profile	The name of the MAP ATI NetworkProfile to be used to build the REST request	String	Any (Input or internal parameter or Constant)	Mandatory
Status	The parameter to store the number portability status from successful REST response	Integer	Internal or output parameter	Mandatory
Default value for Status	The default value to be used for Status if REST request was not successful due to connection/request timeout	Integer	Integer between -2147483648 to 2147483647	Optional
Routing Number	The parameter to store the routing number from successful REST response	String	Internal or output parameter	Mandatory
Default value for Routing Number	The default value to be used for Routeing Number if REST request was not successful due to connection/request timeout	String	Any constant	Optional
IMSI	The parameter to store the IMSI from successful REST response	String	Internal or output parameter	Mandatory
Default value for IMSI	The default value to be used for IMSI if REST request was not successful due to connection/request timeout	String	Any constant	Optional
Fault Code	The parameter to store the fault code from unsuccessful REST response	Integer	Internal or output parameter	Mandatory



5 Appendix 1

5.1 Regular Expressions

The regular expression support provided by the rule builder is built on the Java implementation of Pattern API. For more information refer to javadoc of Pattern .

Following are the supported Java Regular Expressions.

Table 32 Summary of regular-expression constructs

regular-expression constructs	classes	matches
Characters	x	The character x
	\\	The backslash character
	\0n	The character with octal value 0n (0 <= n <= 7)
	\0nn	The character with octal value 0nn (0 <= n <= 7)
	\0mnn	The character with octal value 0mnn (0 <= m <= 3, 0 <= n <= 7)
	\xhh	The character with hexadecimal value 0xhh
	\uhhhh	The character with hexadecimal value 0xhhhh
	\x{h...h}	The character with hexadecimal value 0xh...h (Character.MIN_CODE_POINT <= 0xh...h <= Character.MAX_CODE_POINT)
	\t	The tab character ('\u0009')
	\n	The newline (line feed) character ('\u000A')
	\r	The carriage-return character ('\u000D')
	\f	The form-feed character ('\u000C')



regular-expression constructs	classes	matches
	\a	The alert (bell) character ('\\u0007')
	\e	The escape character ('\\u001B')
	\cx	The control character corresponding to x
Character classes	[abc]	a, b, or c (simple class)
	[^abc]	Any character except a, b, or c (negation)
	[a-zA-Z]	a through z or A through Z, inclusive (range)
	[a-d[m-p]]	a through d, or m through p: [a-dm-p] (union)
	[a-z&&[def]]	d, e, or f (intersection)
	[a-z&&[^bc]]	a through z, except for b and c: [ad-z] (subtraction)
	[a-z&&[^m-p]]	a through z, and not m through p: [a-lq-z] (subtraction)
Predefined character classes	.	Any character (may or may not match line terminators)
	\d	A digit: [0-9]
	\D	A non-digit: [^0-9]
	\s	A whitespace character: [\\t\\n\\x0B\\f\\r]
	\S	A non-whitespace character: [^\\s]
	\w	A word character: [a-zA-Z_0-9]
	\W	A non-word character: [^\\w]
POSIX character classes (US-ASCII only)	\p{Lower}	A lower-case alphabetic character: [a-z]
	\p{Upper}	An upper-case alphabetic character:[A-Z]
	\p{ASCII}	All ASCII:[\\x00-\\x7F]



regular-expression constructs	classes	matches
	\p{Alpha}	An alphabetic character: [\p{Lower}\p{Upper}]
	\p{Digit}	A decimal digit: [0-9]
	\p{Alnum}	An alphanumeric character: [\p{Alpha}\p{Digit}]
	\p{Punct}	Punctuation: One of !"#\$%&'()*+,-./:;<=>? @[\]^_`{ }~
	\p{Graph}	A visible character: [\p{Alnum}\p{Punct}]
	\p{Print}	A printable character: [\p{Graph}\x20]
	\p{Blank}	A space or a tab: [\t]
	\p{Cntrl}	A control character: [\x00-\x1F\x7F]
	\p{XDigit}	A hexadecimal digit: [0-9a-fA-F]
	\p{Space}	A whitespace character: [\t\n\x0B\f\r]
java.lang.Character classes (simple java character type)	\p{javaLowerCase}	Equivalent to java.lang.Character.isLowerCase()
	\p{javaUpperCase}	Equivalent to java.lang.Character.isUpperCase()
	\p{javaWhitespace}	Equivalent to java.lang.Character.isWhitespace()
	\p{javaMirrored}	Equivalent to java.lang.Character.isMirrored()
Classes for Unicode scripts, blocks, categories and binary properties	\p{IsLatin}	A Latin script character (script)
	\p{InGreek}	A character in the Greek block (block)
	\p{Lu}	An uppercase letter (category)
	\p{IsAlphabetic}	An alphabetic character (binary property)



regular-expression constructs	classes	matches
	\p{Sc}	A currency symbol
	\P{InGreek}	Any character except one in the Greek block (negation)
	[\p{L}&&[^\p{Lu}]]	Any letter except an uppercase letter (subtraction)
Boundary matchers	^	The beginning of a line
	\$	The end of a line
	\b	A word boundary
	\B	A non-word boundary
	\A	The beginning of the input
	\G	The end of the previous match
	\Z	The end of the input but for the final terminator, if any
	\z	The end of the input
Greedy quantifiers	X?	X, once or not at all
	X*	X, zero or more times
	X+	X, one or more times
	X{n}	X, exactly n times
	X{n,}	X, at least n times
	X{n,m}	X, at least n but not more than m times
Reluctant quantifiers	X??	X, once or not at all
	X*?	X, zero or more times
	X+?	X, one or more times
	X{n}?	X, exactly n times
	X{n,}?	X, at least n times
	X{n,m}?	X, at least n but not more than m times
Possessive quantifiers	X?+	X, once or not at all
	X*+	X, zero or more times
	X++	X, one or more times



regular-expression constructs	classes	matches
	$X\{n\}^+$	X, exactly n times
	$X\{n,\}^+$	X, at least n times
	$X\{n,m\}^+$	X, at least n but not more than m times
Logical operators	XY	X followed by Y
	$X Y$	Either X or Y
	(X)	X, as a capturing group
Back references	$\backslash n$	Whatever the nth capturing group matched
	$\backslash k<name>$	Whatever the named-capturing group "name" matched
Quotation	$\backslash$	Nothing, but quotes the following character
	$\backslash Q$	Nothing, but quotes all characters until $\backslash E$
	$\backslash E$	Nothing, but ends quoting started by $\backslash Q$
Special constructs (named-capturing and non-capturing)	$(?<name>X)$	X, as a named-capturing group
	$(?:X)$	X, as a non-capturing group
	$(?idmsuxU-idmsuxU)$	Nothing, but turns match flags i d m s u x U on - off
	$(?idmsux-idmsux:X)$	X, as a non-capturing group with the given flags i d m s u x on - off
	$(?=X)$	X, via zero-width positive lookahead
	$(?!X)$	X, via zero-width negative lookahead
	$(?<=X)$	X, via zero-width positive lookbehind
	$(?<!X)$	X, via zero-width negative lookbehind
	$(?>X)$	X, as an independent, non-capturing group



Example 1

To treat the above patterns as a character/string for string operations, prefix it with an escape character (\) as shown in the below example.

The following table shows an example of SplitStringModifier behavior for a delimiter containing regex pattern (.) and its corresponding escaped value (\.) producing different output values for the same input value.

Input value	Constant to Delimit	Output value
192.1.111.8	.	192.1.111.8
192.1.111.8	\.	[192,1,111,8]